

```
def increase_speed(self):
    """Increase speed settings and alien point values."""
    self.ship_speed *= self.speedup_scale
    self.bullet_speed *= self.speedup_scale
    self.alien_speed *= self.speedup_scale
```

```
② self.alien_points = int(self.alien_points * self.score_scale)
```

We define a rate at which points increase, which we call `score_scale` ❶. A small increase in speed (1.1) makes the game more challenging quickly. But to see a more notable difference in scoring, we need to change the alien point value by a larger amount (1.5). Now when we increase the game's speed, we also increase the point value of each hit ❷. We use the `int()` function to increase the point value by whole integers.

To see the value of each alien, add a `print()` call to the `increase_speed()` method in `Settings`:

```
settings.py    def increase_speed(self):
               --snip--
               self.alien_points = int(self.alien_points * self.score_scale)
               print(self.alien_points)
```

The new point value should appear in the terminal every time you reach a new level.

NOTE

Be sure to remove the `print()` call after verifying that the point value is increasing, or it might affect your game's performance and distract the player.

Rounding the Score

Most arcade-style shooting games report scores as multiples of 10, so let's follow that lead with our scores. Also, let's format the score to include comma separators in large numbers. We'll make this change in `Scoreboard`:

```
scoreboard.py def prep_score(self):
               """Turn the score into a rendered image."""
               rounded_score = round(self.stats.score, -1)
               score_str = f"{rounded_score:,}"
               self.score_image = self.font.render(score_str, True,
               self.text_color, self.settings.bg_color)
               --snip--
```

The `round()` function normally rounds a float to a set number of decimal places given as the second argument. However, when you pass a negative number as the second argument, `round()` will round the value to the nearest 10, 100, 1,000, and so on. This code tells Python to round the value of `stats.score` to the nearest 10 and assign it to `rounded_score`.

We then use a format specifier in the f-string for the score. A *format specifier* is a special sequence of characters that modifies the way a variable's value is presented. In this case the sequence `:,` tells Python to insert